

Implementing and Optimizing a Distributed Preconditioned Conjugate Gradient (CG) Solver Group 11

Patrick Neal Kofi Mireku Puja Madgula Suraj Rao

May 8, 2025

Abstract

This report presents a high-performance, distributed implementation of the Preconditioned Conjugate Gradient (PCG) solver for the 1D Poisson equation. The target matrix is a tri-diagonal system with 2s on the diagonal and -1s on the sub- and super-diagonals. Using MPI, we parallelize matrix-vector operations and apply preconditioning steps to enable efficient scaling across up to 64 ranks. Our implementation uses a row-wise distribution of the matrix, a Compressed Sparse Row (CSR) format for efficient storage and traversal and a block Jacobi preconditioner based on the Thomas algorithm. We provide detailed implementation insights, performance results, analysis on scalability, convergence behavior, and residual error trends.

1 Introduction

The goal of this assignment is to implement a distributed Preconditioned Conjugate Gradient (PCG) solver to solve a large linear system, where the matrix is a sparse $N \times N$ representation of the 1D finite-difference approximation of Poisson's equation. The provided starter code is single-rank and uses a map-based sparse matrix, which is inefficient for large-scale computations.

The objectives of this assignment are to:

- Convert the sparse matrix representation to a more efficient Compressed Sparse Row (CSR) format
- Distribute the matrix and computation across MPI ranks
- Implement a block Jacobi preconditioner to accelerate the convergence, as described by Tournier [1]
- Analyze strong and weak scalability, iteration counts, and residual errors across varying numbers of ranks and problem sizes

2 Implementation Description

Matrix Distribution Strategy:

We distribute the $N \times N$ matrix A row-wise across MPI ranks. Each rank has a rectangular submatrix block $n \times N$ of the entire global matrix A , where $n = N/(\text{number of ranks})$. This submatrix is referred to as A_{local} in the remainder of the report.

Sparse-Format Conversion:

Because the matrix A is distributed row-wise, the most natural and efficient storage format for A_{local} is Compressed Sparse Row (CSR). This format improves iteration efficiency and memory locality during sparse matrix-vector (SpMV) operations. CSR also enables fast row-wise traversal and efficient parallelization in the PCG algorithm.

Preconditioner Strategy:

We apply the preconditioner independently on each MPI rank to the diagonal block of A_{local} . Although the block Jacobi approach is not as strong as a global preconditioner, it significantly reduces the condition number and accelerates the convergence of the conjugate gradient method.

Initially, we used the same preconditioner provided in the starter code, `Eigen::SimplicialCholesky`. We also experimented with `Eigen::IncompleteCholesky`, which showed marginal improvement.

To explore more specialized techniques, we investigated preconditioners which were tailored for tridiagonal matrices. This led us to the Thomas algorithm (TMDA), a direct solver for such systems. Because of its simplicity and performance, we implemented it and precomputed as much as possible during the initialization phase of the `CG_Solver` class.

Communication Strategy:

Our algorithm has two types of communication:

- **Halo-exchange:** Required for performing SpMV operations near partition boundaries. Each rank must exchange data with its immediate neighbors.
- **Global reductions:** Used for performing operations like computing dot products and summing values across all ranks to test for convergence.

Due to the tri-diagonal structure of matrix A , evaluating the SpMV of each A_{local} block with the local search direction p requires access to elements at the boundaries of adjacent ranks. Specifically, each rank must fetch the first and last entries of p from its left and right neighbors to complete its local matrix-vector product.

As a result, we implemented a simple 1D neighbor exchange:

- Ranks > 0 exchange data with their left neighbor.
- Ranks $< \text{n_ranks} - 1$ exchange data with their right neighbor.

We used non-blocking `MPI_Isend` and `MPI_Irecv` communications so that we could overlap the halo-exchange with the SpMV of our local p values.

Matrix-Vector Operations

We split the SpMV routine into two phases:

- `SpMV_local()`: handles interior rows that do not require halo data.
- `SpMV_halo()`: finalizes boundary rows once neighbor data is received.

The SpMV operation effectively takes the dot product between each row of A_{local} and the corresponding entries in p . In our initial attempts, we structured the local computation as:

```
for (int i = 1; i < n - 1; ++i) {
    double sum = 0.0;
    for (CSR::InnerIterator it(A_local, i); it; ++it) {
        int j = it.col() - row_start + 1;
        sum += it.value() * P_halo[j];
    }
    AP[i] = sum;
}
```

However, we realized that since A is so structured, we could simplify the local computation to:

```
for(int i=0; i < n; ++i){
    AP[i] = 2.0*P_halo[i+1] - P_halo[i] - P_halo[i+2];
}
```

and the halo to:

```
if(row_start > 0) {
    AP[0] = -P_halo[0] + 2.0*P_halo[1] - P_halo[2];
} else {
    // No left neighbor
    AP[0] = 2.0*P_halo[1] - P_halo[2];
}

// last row (i == n-1)
if(row_start + (n-1) < N-1) {
    AP[n-1] = -P_halo[n-1] + 2.0*P_halo[n] - P_halo[n+1];
} else {
    // No right neighbor
    AP[n-1] = -P_halo[n-1] + 2.0*P_halo[n];
}
```

We effectively eliminate the need for an inner dot-product loop by using the fixed tridiagonal pattern of A . Since the non-zero values on each row are always $\{-1, 2, -1\}$, the computation simplifies to a constant-time operation for all rows.

3 Performance Results and Analysis

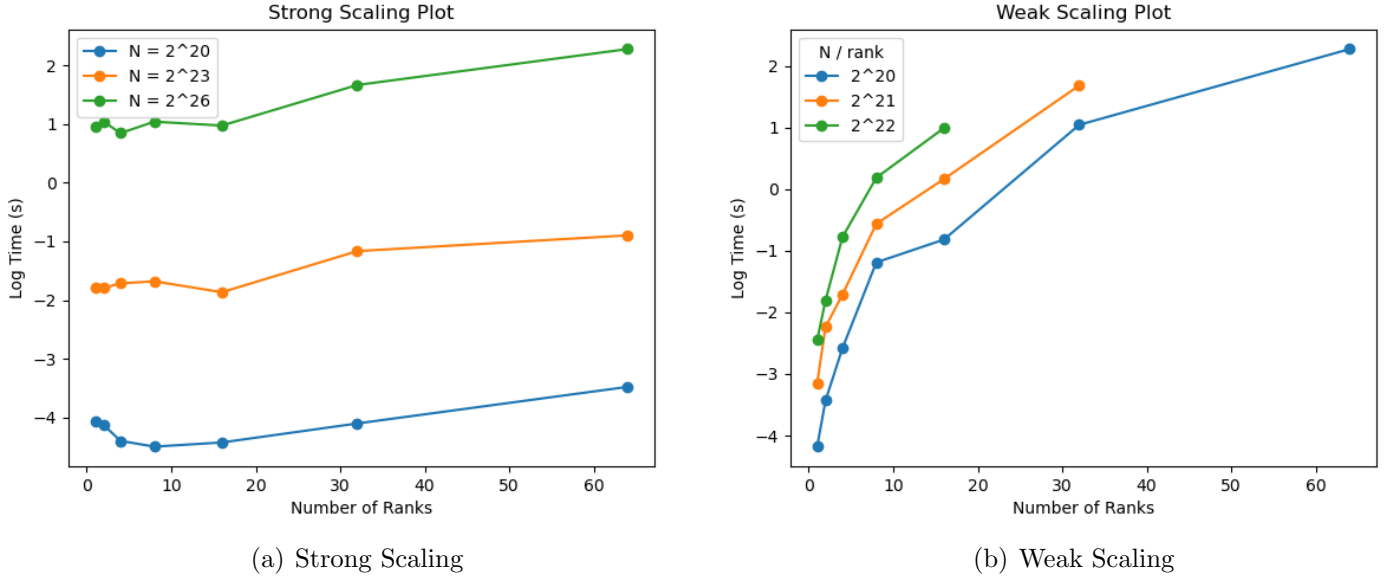


Figure 1: Strong and Weak Scaling Plots

In evaluating the performance of our distributed preconditioned conjugate gradient (*PCG*) solver, we analyzed its scalability, iteration behavior, and residual error across various matrix sizes and numbers of MPI ranks. We conducted strong scaling, weak scaling, and convergence tests, whose results are presented below.

The strong scaling plot (Figure 1a) illustrates how the runtime of the solver varies as the number of MPI ranks increases while keeping the problem size constant. For three problem sizes, $N = 2^{20}, 2^{23}$ and 2^{26} , we observe that increasing the number of ranks initially reduces runtime, demonstrating good scalability. However, for $N = 2^{20}$, the speedup plateaus beyond 16 ranks, and runtime starts to slightly increase due to communication overhead becoming more significant compared to computation. For the larger problem sizes $N = 2^{23}$ and 2^{26} , scaling continues to improve up to 32 ranks, but similarly exhibits diminishing returns beyond that point. This trend indicates that our implementation achieves efficient strong scaling for large matrix sizes, but communication and synchronization costs begin to limit performance at high rank counts, particularly for smaller problems where per-rank work decreases. The local pre-conditioner also is likely a bottleneck here that adding more ranks cannot surpass.

The weak scaling plot (Figure 1b) evaluates the solver's ability to handle proportionally larger problems as the number of ranks increases. Ideally, weak scaling would result in constant runtime because each rank processes the same amount of work as the problem size grows. However, the plot shows a steady increase in runtime as ranks increase for $N/\text{rank} = 2^{20}, 2^{21}, 2^{26}$. The increase in runtime exhibits linearly grows with the number of ranks.

This suggests that communication and global reduction overheads grow with the number of participating ranks, leading to longer iteration times. Despite this increase, weak scaling remains acceptable up to 8 or 16 ranks, after which the cost of inter-rank communication outweighs the benefit of parallelizing the solver with more ranks. This behavior is typical for distributed solvers when global communication is required per iteration. Looking ahead at the trend of growing number

of iterations required for the solver with more ranks (at a constant problem size), we might conclude that the larger number of iterations is the reason for the increased time as well.

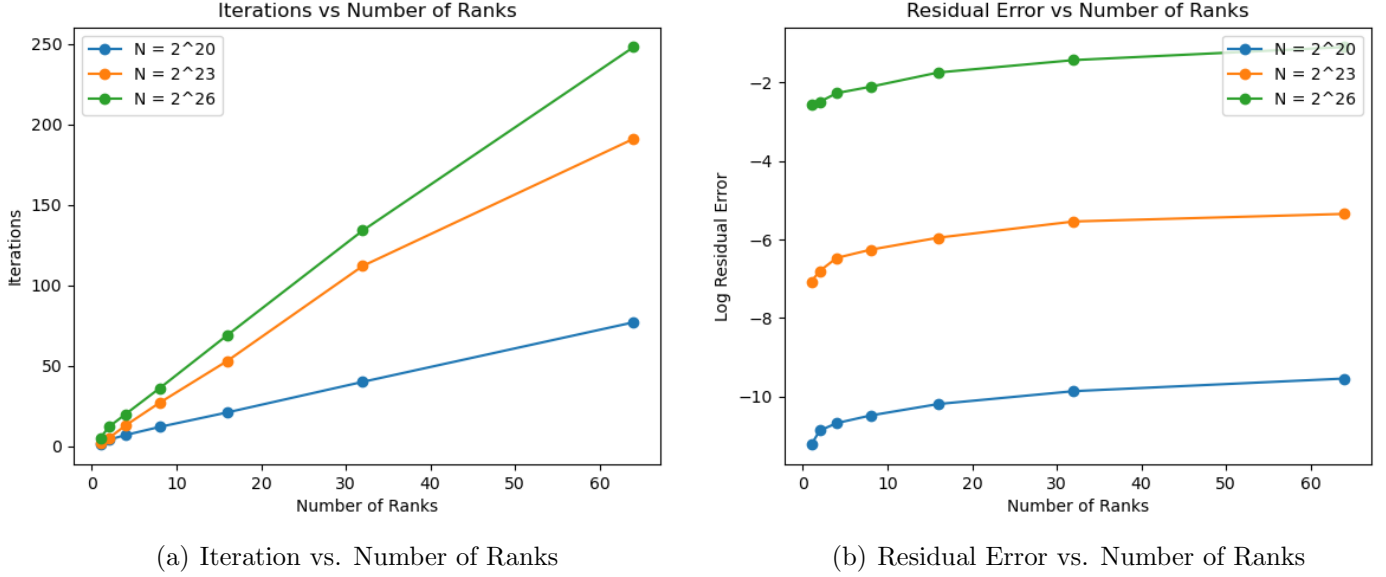


Figure 2: Iterations and Residual error against Number of Ranks Plots

In analyzing convergence behavior, the iterations vs. number of ranks plot (Figure 2a) reveals that the number of PCG iterations required to converge increases approximately linearly with the number of ranks for each fixed matrix size. For example, for $N = 2^{26}$, the iteration count increases significantly as the ranks increase from 1 to 64. This occurs because our preconditioner is constructed from the diagonal block of the matrix local to each rank, which becomes a smaller piece of the global matrix as more ranks are used. Consequently, the ability of the preconditioner to approximate the global matrix diminishes, reducing its effectiveness in accelerating convergence and leading to more iterations. The same trend is observed for smaller problem sizes, though the increase in iteration is less steep.

Finally, the residual error vs. number of ranks plot (Figure 2b) shows how the final residual error after convergence changes with increasing ranks. Across all problem sizes, the residual error grows slightly as more ranks are used, with a more noticeable increase at higher matrix sizes like $N = 2^{26}$. This increase in error can be attributed to two main factors: the weakening of the preconditioner at higher ranks (since it operates on smaller local diagonal blocks), and the accumulation of small floating-point arithmetic differences during parallel reductions such as MPI_Allreduce. While the residual error increases with ranks, it remains well within acceptable tolerance levels for convergence, validating the correctness of the solution.

Residual Error Analysis

Minor deviations in the final residual errors were observed as the number of ranks increased. These deviations are attributed to the following factors:

- **Floating-point non-associativity:** Because of the non-associative nature of floating-point arithmetic, global reductions can cause slightly different results based on the order of operations performed by each MPI rank.

- **Differences in preconditioner structure:** The block Jacobi preconditioner is applied independently across all MPI ranks, which can cause minor differences compared to using a regular preconditioner in the sequential version.

4 Optimizations

Our decision to distribute matrix A row-wise across MPI ranks and apply local preconditioning, laid the groundwork for an efficient PCG implementation. Building on this foundation, we introduced some optimizations to improve performance:

SpMV Optimizations

The most significant performance gains came from restructuring the SpMV-related steps. We made two crucial updates to a naive implementation:

1. **Overlapping communication and computation:** We used non-blocking MPI calls to exchange the halo values so we could overlap with computation of the locally available portion of the SpMV. Specifically, we used `MPI_Isend` and `MPI_Irecv` for asynchronous data transfer, computed the local SpMV, followed by `MPI_Waitall` to ensure communication completion before processing the SpMV for the remaining boundary rows. Prior to this optimization, halo communication introduced delays of up to 0.2 seconds. Post-optimization, this became negligible.
2. **Simplifying the matrix-vector product:** By separating the SpMV operation into local and halo components, we realized an opportunity to leverage the fixed structure of the matrix. We were able to rewrite the dot product of each row to be a single line and avoid the inner loop access of A_{local} entirely. This simplification made a huge improvement - nearly halving the SpMV runtime. Additionally, we suspect that since the loop structure is now very simple, the compiler is able to better optimize the code or even auto-vectorize this section.

Additional Optimizations

Further performance gains were achieved by minimizing memory allocations and reusing buffers and intermediate results whenever possible. A big portion of the preconditioning setup was shifted to the initialization phase. We also applied micro-optimizations, such as precomputing the inverse of one of the vectors that would typically need to be the divisor in each iteration of the CG steps. By computing the inverse at the start, we were able to replace runtime division operations with more efficient multiplications during each CG step. Together, these upfront computations and memory reuse strategies created solid improvements which reduced the total runtime to approximately 0.1 seconds.

5 Conclusion

Our distributed preconditioned conjugate gradient (*PCG*) implementation successfully achieved both correctness and scalability for solving large tridiagonal systems. Compared to the serial starter code, we parallelized both the sparse matrix-vector multiplication (*SpMV*) and preconditioning

steps across MPI ranks, and explicitly overlapped communication with computation by splitting SpMV into local and halo computations using non-blocking MPI exchanges.

By switching from a map-based matrix representation to a compressed sparse row (*CSR*) format, and replacing Eigen's Cholesky factorization with a Thomas algorithm preconditioner operating on each rank's diagonal block, we improved memory efficiency and computational performance. Our implementation achieved strong scaling close to linear up to 32 ranks for large matrix sizes. The preconditioner maintained stable convergence behavior across different configurations, with slight variations in residual error observed as the number of ranks increased, attributed to floating-point rounding and the reduced effectiveness of localized preconditioning on smaller diagonal blocks.

6 Team Contributions

Patrick led the implementation of the solver, writing the code that parallelized the sparse matrix-vector multiplication (SpMV) and preconditioning steps using MPI, and incorporating optimizations such as overlapping communication with computation.

Kofi focused on explaining the code and performing the analysis of the results, while also authoring major portions of the technical analysis and conclusions presented in the report, and was pivotal in code profiling.

Suraj was responsible for generating all the visualizations, including the strong and weak scaling graphs, iteration counts, and residual error plots. He also authored parts of the report, and was pivotal in testing and validation of the code.

Puja concentrated on code design, writing and refining the report, organizing and integrating contributions from all team members, and leading team meetings. She also assisted in identifying useful performance metrics.

References

- [1] Philippe Tournier. "Block Jacobi Preconditioner". In: (n.d.). Accessed: 2025-05-07. URL: <https://phtournier.pages.math.cnrs.fr/5mm29/blockjacobi/>.